Ex No:7	Interactive Graph Visualization
Date:-20/03/26	Video Link - https://youtu.be/RNVAmW_fXuc

Aim

To design and implement an interactive graph visualization system using Python that allows users to perform basic graph operations such as adding, deleting, and searching nodes and edges.

Tools and Technologies Used:

o Programming Language: Python 3

o Environment: Jupyter Notebook.

o Tkinter (GUI)

o NetworkX (Graph Library)

o Matplotlib (Visualization)

Description

Graphs are an important data structure used to represent relationships between objects. In this experiment, an interactive application is developed using:

• Tkinter for creating the graphical user interface (GUI)

• NetworkX for graph creation and manipulation

• Matplotlib for visualizing the graph

The system allows users to dynamically modify the graph structure through GUI buttons and visualize the updated graph instantly.

List of tasks

1. Display the Graph

Visualize the graph using a suitable layout algorithm to clearly represent nodes and edges.

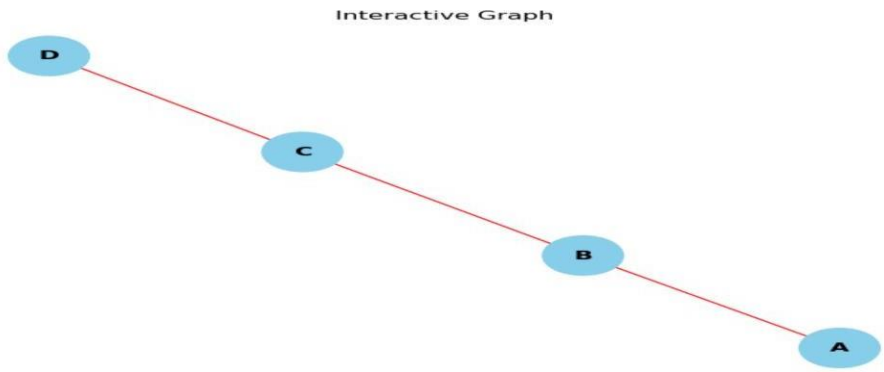
Code

```
import tkinter as tk
from tkinter import simpledialog
import networkx as nx
import matplotlib.pyplot as plt
#create graph data
```

```
def create_graph():
    G=nx.Graph()
    G.add_edges_from([('A','B'),('B','C'),('C','D')])
    return G
#visualize graph
def visualize_graph(G):
    try:
        print(f"Nodes:{G.nodes()}")
        print(f"Edges:{G.edges()}")
        pos=nx.spring_layout(G)
        plt.figure(figsize=(8,8))
        nx.draw(G,with_labels=True, node_color='skyblue',
node_size=2000,font_size=16,font_weight='bold',edge_color='gray')
        plt.title("Interactive Graph Visualization")
        plt.show()
    except Exception as e:
        print(f"Error: {e}")
```

Output:-

```
Nodes:['A', 'B', 'C', 'D']
Edges:[('A', 'B'), ('B', 'C'), ('C', 'D')]
```



2. Design the GUI

Create a graphical user interface using Tkinter. Provide buttons for user interaction with the graph.

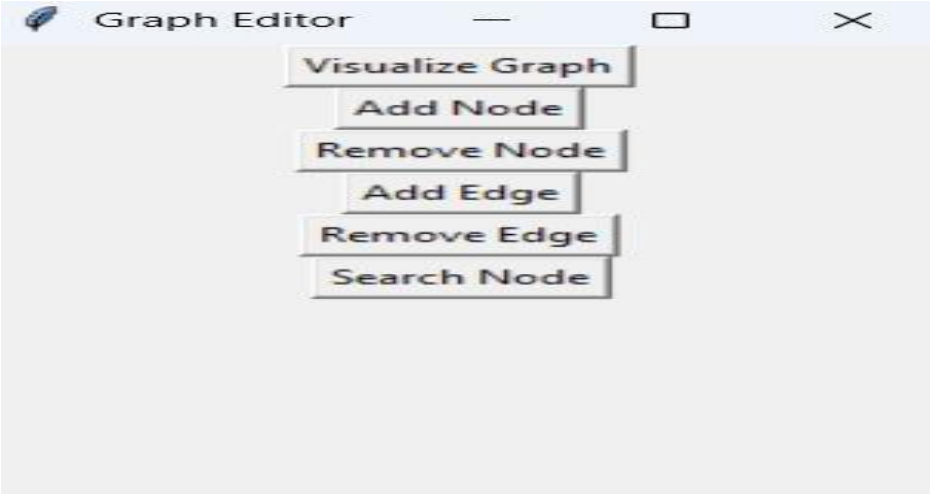
Code

```
#gui
def create_gui():
```

```
    global root
      root=tk.Tk()
      root.title("Interactive Graph visualization")
      G=create_graph()
      tk.Button(root,text="Visualize Graph", command=lambda: visualize_graph(G)).pack()
      tk.Button(root,text="Add node", command=lambda: add_node(G)).pack()
      tk.Button(root,text="Add edge", command=lambda: add_edge(G)).pack()
      tk.Button(root,text="Remove node", command=lambda: remove_node(G)).pack()
      tk.Button(root,text="Remove edge", command=lambda: remove_edge(G)).pack()
      tk.Button(root,text="search node", command=lambda: search_node(G)).pack()
      root.mainloop()

  if __name__=="__main__":
      create_gui()
```

Output:-



3. Add a Node

Allow the user to insert a new node into the graph dynamically.

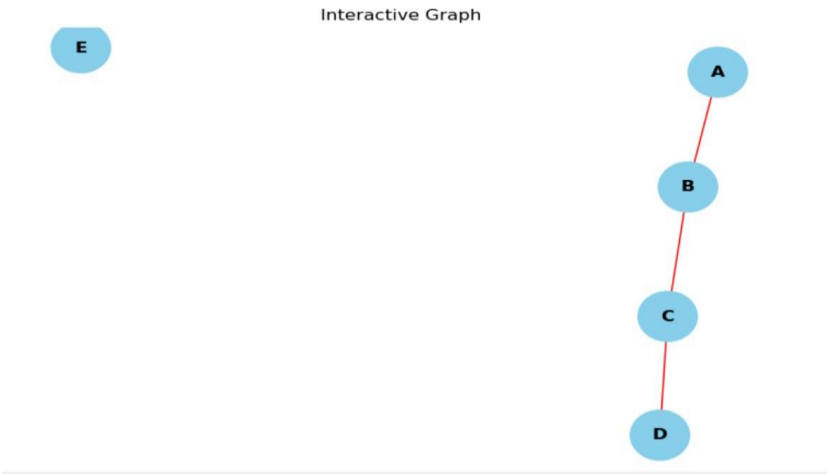
Code

```
#Add Node
```

```
def add_node(G):
    new_node=simpledialog.askstring("Input","Enter new node:",parent=root)
    if new_node:
        G.add_node(new_node)
        print(f"Added node{new_node}")
        visualize_graph(G)
```

Output:-

```
Added nodeE
Nodes:['A', 'B', 'C', 'D', 'E']
Edges:[('A', 'B'), ('B', 'C'), ('C', 'D')]
```



4. Add an Edge

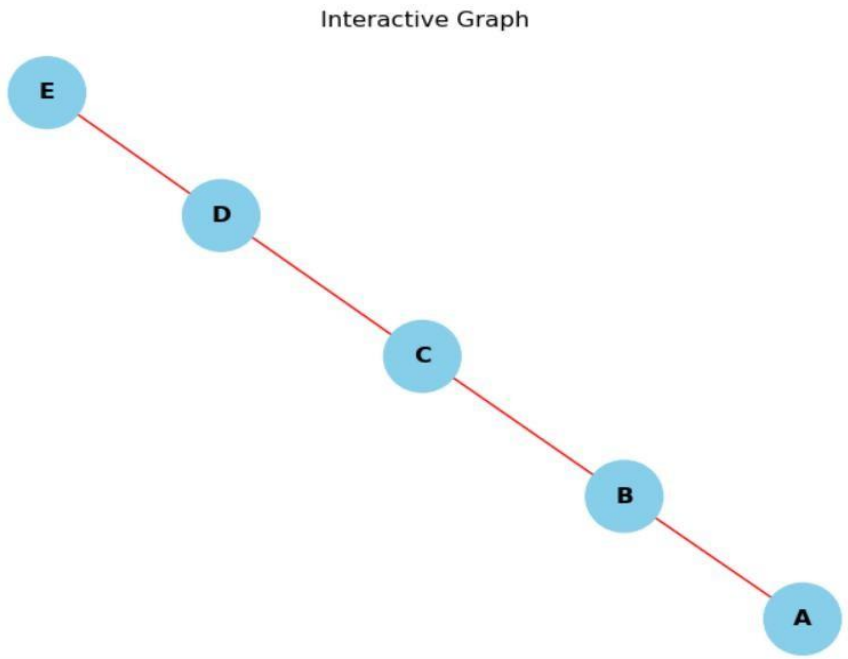
Allow the user to create a connection between two nodes.

Code

```
#Add edge
def add_edge(G):
    node1=simpledialog.askstring("Input","enter first node:",parent=root)
    node2=simpledialog.askstring("Input","enter second node:",parent=root)
    if node1 and node2:
        G.add_edge(node1,node2)
        print(f"Added edge {node1}-{node2}")
```

```
visualize_graph(G)
```

Output:-



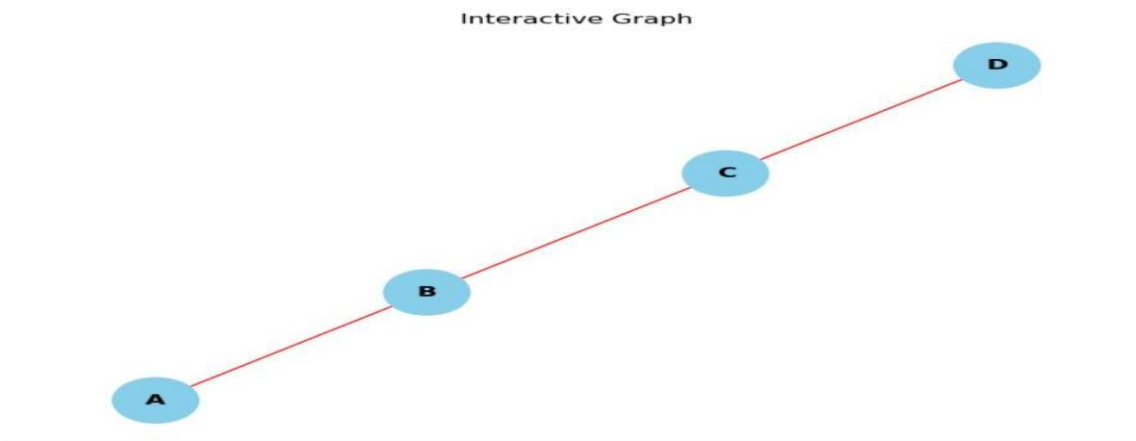
5. Remove a Node

Enable deletion of a selected node along with all its associated edges.

Code

```
# Remove Node
def remove_node(G):
    node = simpledialog.askstring("Input", "Enter node to remove:", parent=root)
    if node:
        if node in G:
            G.remove_node(node)
            print(f"Removed node {node}")
            visualize_graph(G)
        else:
            print(f"Node {node} does not exist")
```

Output:-



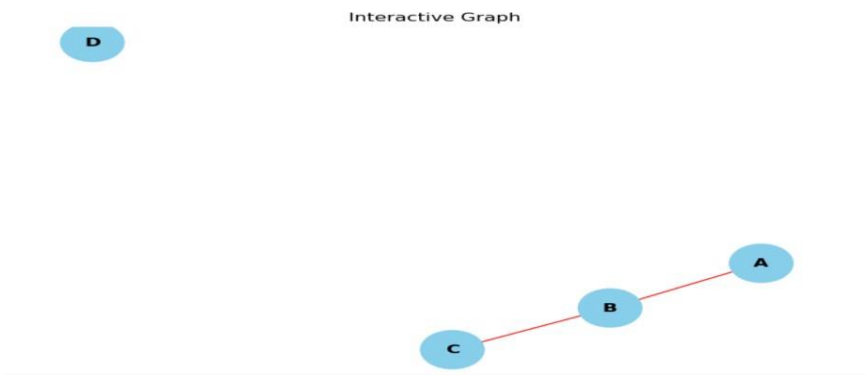
6. Remove an Edge

Allow deletion of a connection between two specified nodes.

Code

```
def remove_edge(G):
    node1 = simpledialog.askstring("Input", "Enter first node:", parent=root)
    node2 = simpledialog.askstring("Input", "Enter second node:", parent=root)
    if node1 and node2:
        if G.has_edge(node1, node2):
            G.remove_edge(node1, node2)
            print(f"Removed edge {node1}-{node2}")
            visualize_graph(G)
        else:
            print(f"Edge {node1}-{node2} does not exist")
```

Output:-



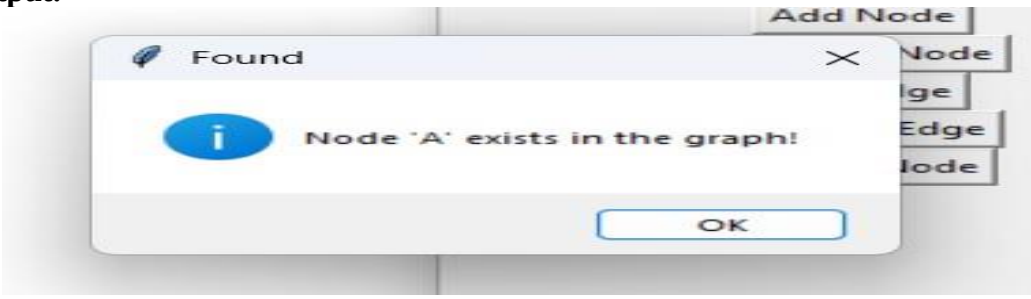
7. Search for a Node

Check whether a given node exists in the graph and display the result.

Code

```
# Search Node
def search_node(G):
    node = simpledialog.askstring("Input", "Enter node to search:", parent=root)
    if node:
        if node in G:
            print(f"Node {node} exists in the graph")
        else:
         print(f"Node {node} not found in the graph")
```

Output:-



Result: Based on Interactive Graph Visualization the programs have been executed and the outputs have been verified successfully.